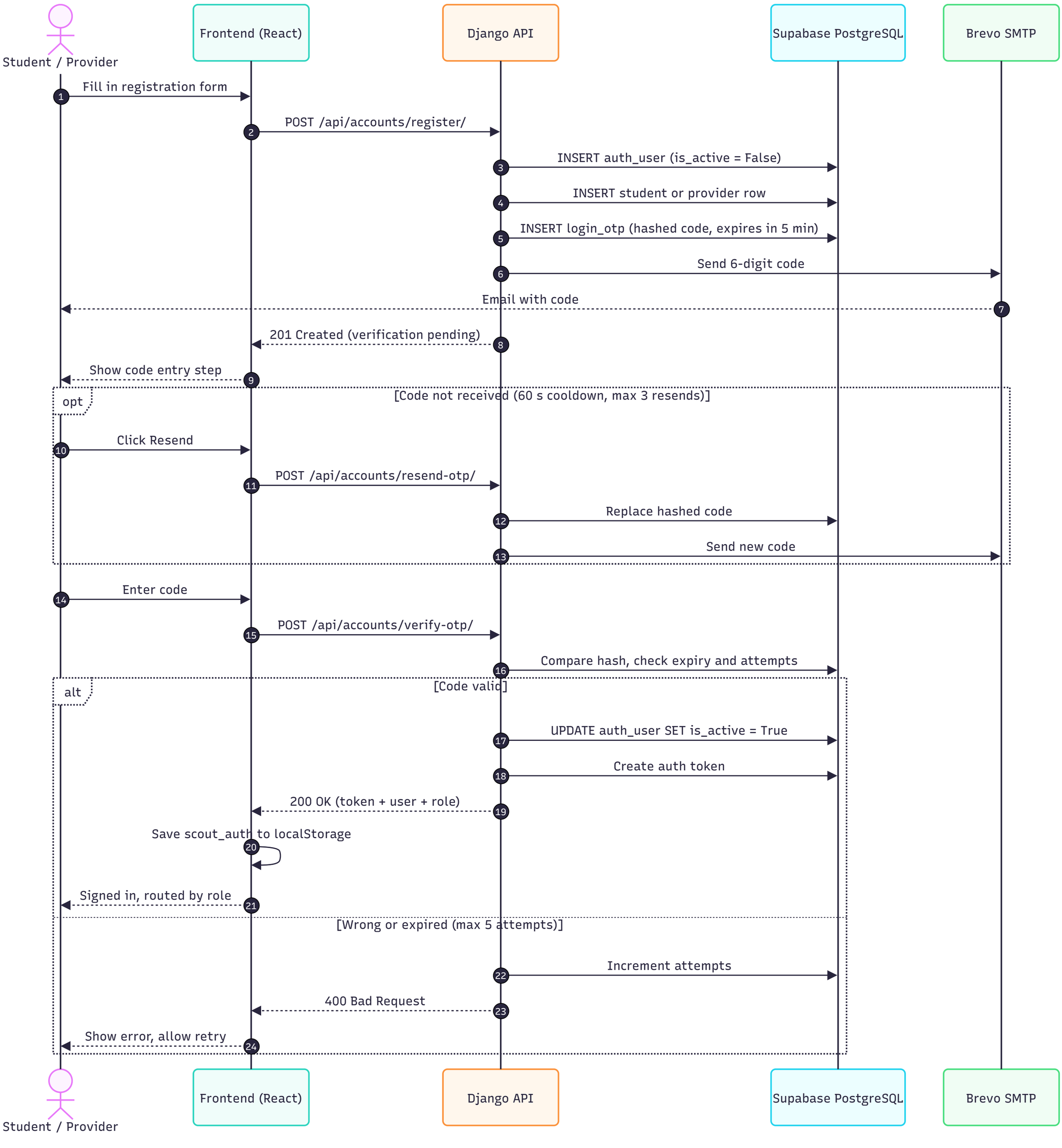
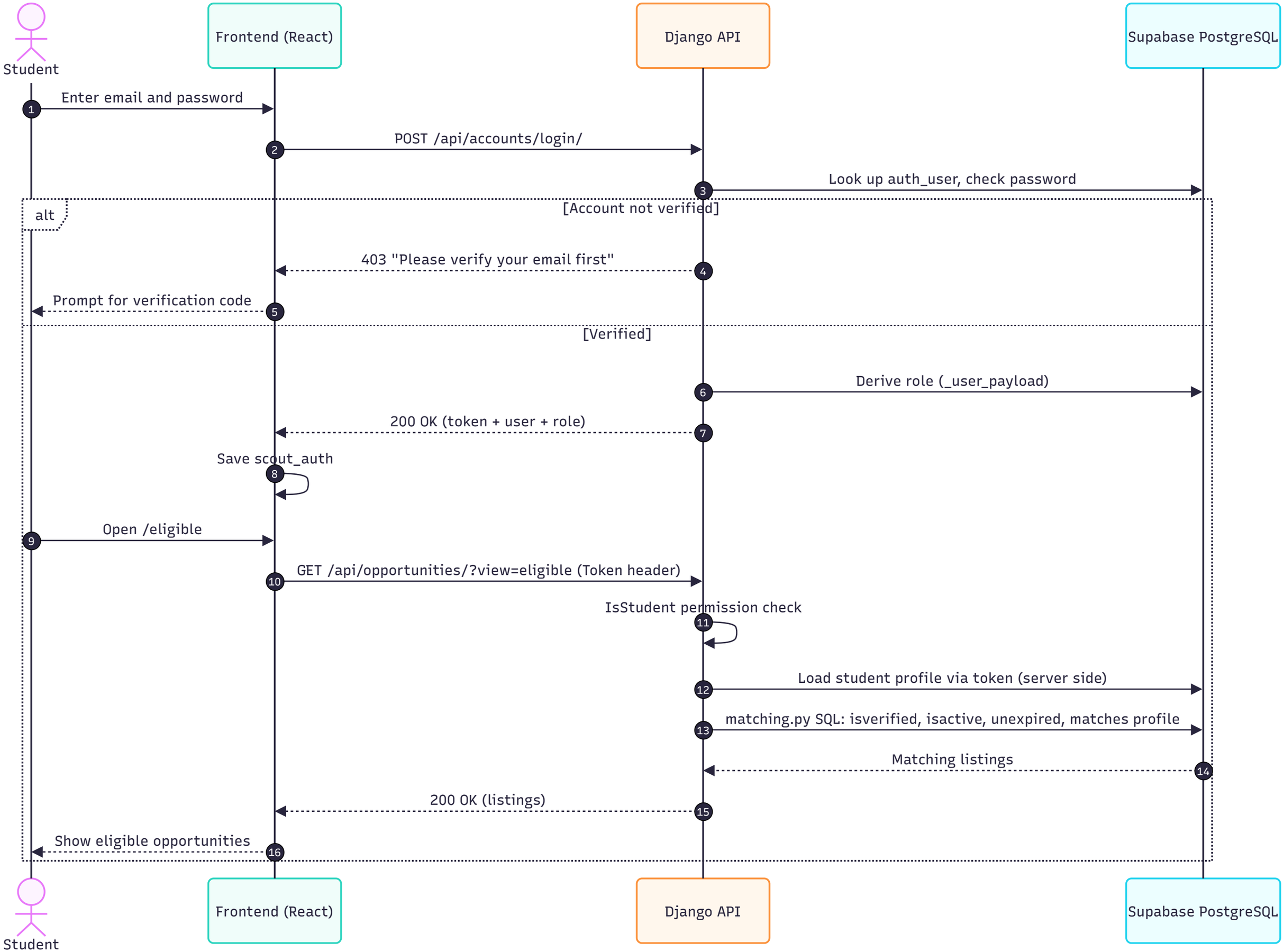
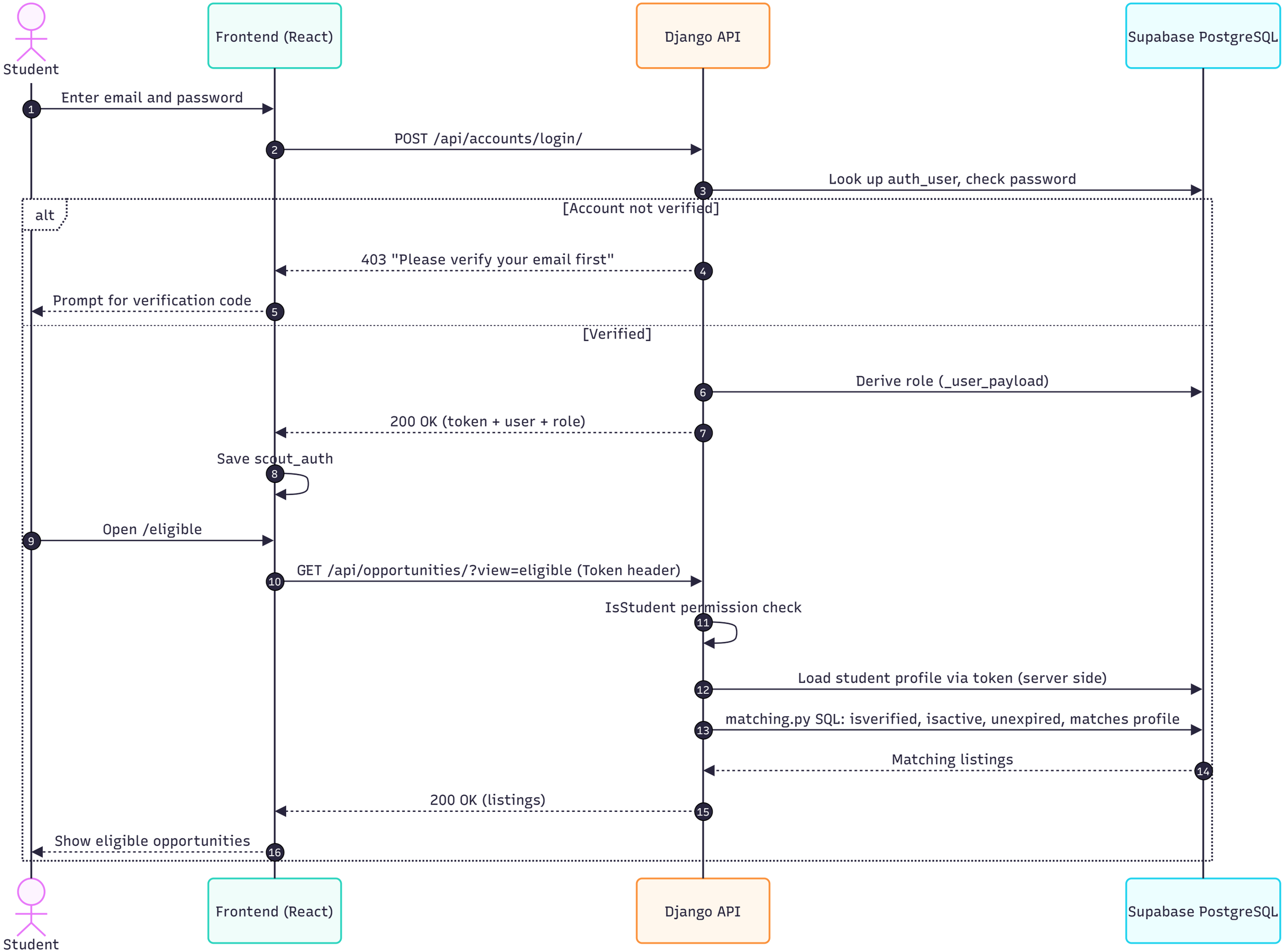
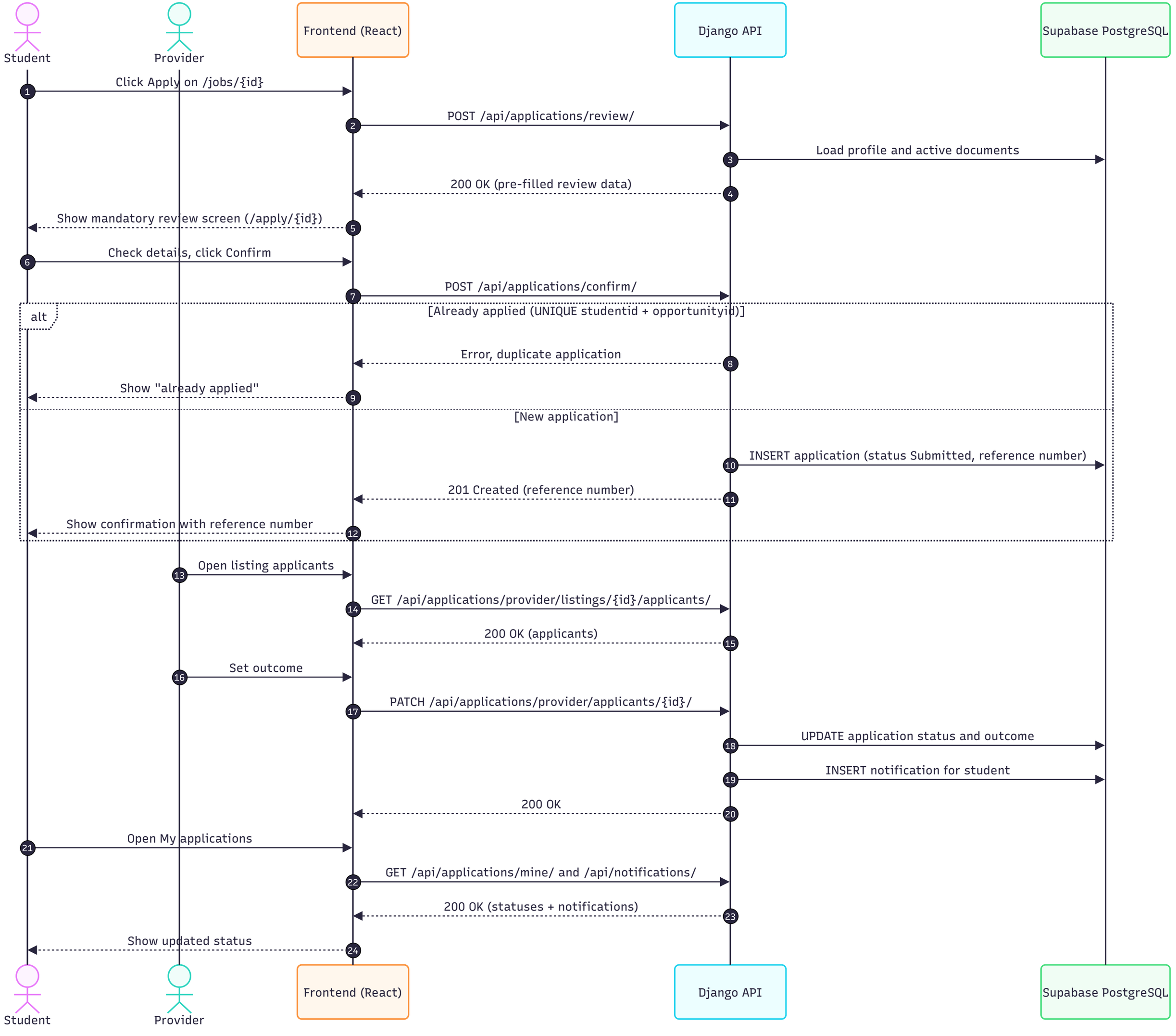

Student / Provider
Frontend (React)
Django API
Supabase PostgreSQL
Brevo SMTP
1 Fill in registration form
2 POST /api/accounts/register/
3 INSERT auth_user (is_active = False)
4 INSERT student or provider row
5 INSERT login_otp (hashed code, expires in 5 min)
6 Send 6-digit code
7 Email with code
8 201 Created (verification pending)
9 Show code entry step
opt
[Code not received (60 s cooldown, max 3 resends)]
10 Click Resend
11 POST /api/accounts/resend-otp/
12 Replace hashed code
13 Send new code
14 Enter code
15 POST /api/accounts/verify-otp/
16 Compare hash, check expiry and attempts
alt
[Code valid]
17 UPDATE auth_user SET is_active = True
18 Create auth token
19 200 OK (token + user + role)
20 Save scout_auth to localStorage
21 Signed in, routed by role
[Wrong or expired (max 5 attempts)]
22 Increment attempts
23 400 Bad Request
24 Show error, allow retry
Student / Provider
Frontend (React)
Django API
Supabase PostgreSQL
Brevo SMTP


Student
Frontend (React)
Django API
Supabase PostgreSQL
1
Enter email and password
2
POST /api/accounts/login/
3
Look up auth_user, check password
alt
[Account not verified]
403 "Please verify your email first"
4
Prompt for verification code
5
[Verified]
6
Derive role (_user_payload)
200 OK (token + user + role)
7
Save scout_auth
8
9
Open /eligible
10
GET /api/opportunities/?view=eligible (Token header)
IsStudent permission check
11
12
Load student profile via token (server side)
13
matching.py SQL: isverified, isactive, unexpired, matches profile
Matching listings
14
200 OK (listings)
15
Show eligible opportunities
16
Frontend (React)
Django API
Supabase PostgreSQL
Student


Student
Frontend (React)
Django API
Supabase PostgreSQL
1
Enter email and password
2
POST /api/accounts/login/
3
Look up auth_user, check password
alt
[Account not verified]
403 "Please verify your email first"
4
Prompt for verification code
5
[Verified]
6
Derive role (_user_payload)
200 OK (token + user + role)
7
Save scout_auth
8
9
Open /eligible
10
GET /api/opportunities/?view=eligible (Token header)
IsStudent permission check
11
12
Load student profile via token (server side)
13
matching.py SQL: isverified, isactive, unexpired, matches profile
Matching listings
14
200 OK (listings)
15
Show eligible opportunities
16
Student
Frontend (React)
Django API
Supabase PostgreSQL


Student
Provider
Frontend (React)
Django API
Supabase PostgreSQL
1 Click Apply on /jobs/{id}
2 POST /api/applications/review/
3 Load profile and active documents
4 200 OK (pre-filled review data)
5 Show mandatory review screen (/apply/{id})
6 Check details, click Confirm
7 POST /api/applications/confirm/
alt
[Already applied (UNIQUE studentid + opportunityid)]
8 Error, duplicate application
9 Show "already applied"
[New application]
10 INSERT application (status Submitted, reference number)
11 201 Created (reference number)
12 Show confirmation with reference number
13 Open listing applicants
14 GET /api/applications/provider/listings/{id}/applicants/
15 200 OK (applicants)
16 Set outcome
17 PATCH /api/applications/provider/applicants/{id}/
18 UPDATE application status and outcome
19 INSERT notification for student
20 200 OK
21 Open My applications
22 GET /api/applications/mine/ and /api/notifications/
23 200 OK (statuses + notifications)
24 Show updated status
Student
Provider
Frontend (React)
Django API
Supabase PostgreSQL